SYSTEM DESIGN • v1.0 • OCT 2026 • DO NOT MODIFY CODE WITHOUT REVIEW

DC
SYSTEM

DISTRICT CAR

Production-Ready Ride / Vehicle Booking Platform

COMPLETE SYSTEM DESIGN

And Interface Architecture — All Connections, All Layers, All Flows

Flutter 3.10+ • Firebase (Auth / Firestore / Functions / FCM / Storage) • Dart Monorepo: customer_app + driver_app + admin_panel + shared + functions

3 Apps Customer + Driver + Admin	**5** Firebase Services Auth • Firestore • Functions • FCM • Storage	**1** Shared Package Models • Theme • Services • Widgets	**0** Surge Server fare • Peer-to-peer cash/UPI

October 2026 • System Architect Edition • For Engineering, Product, Design & Ops

districtcar.in • +91 87896 01387 • Nichey Chowk, Khunti — 835210

Monorepo: district_car / (customer_app, driver_app, admin_panel, shared, functions, firebase)

How to use this document: Engineering → Sections 2-9 (arch, data, APIs, security) | **Design / Product** → Sections 10-11 (UI screens + flows) | **Ops / Marketing** → Sections 1, 12-14 (infra, launch, roadmap). All diagrams are vector — zoom to 400% and stay crisp. No code was modified to produce this document.

CONTENTS

01	Executive Summary — What District Car Is	3
02	Monorepo & Tech Stack — The Five Repos That Ship	3
03	High-Level Architecture — The Big Picture	4
04	Component & Service Map — Who Talks to Whom	5
05	Data Architecture — Firestore Collections & Rules	6
06	Booking State Machine — Server-Authoritative Truth	8
07	Sequence Diagrams — Booking Flows End-to-End	9
08	API & Cloud Functions Interface — The Server Contract	10
09	Location & Trust Engine — Geohash, Haversine, Fares, Photos	11
10	UI/UX Interface Design — Customer, Driver, Admin Screens	12
11	Interface Specifications — Widgets, Themes, Shared Package	13
12	Integration Layer — FCM, Maps, UPI, Monetag, Play Store	14
13	Infrastructure & Deployment — Firebase, Netlify, CI/CD	15
14	Security, Compliance & Offline Strategy	15
15	Non-Functional Requirements & Future Roadmap	16
—	Appendix — Collections Cheat-Sheet, Rates & Contacts	16

Legend: **Blue** = Customer | **Teal** = Driver | **Amber** = Admin | **Purple** = Cloud Functions | **Red** = FCM / Push | **Navy** = Firebase Core

01 — EXECUTIVE SUMMARY — WHAT DISTRICT CAR IS

District Car is an **enterprise-grade, production-ready ride-hailing + light logistics platform** for Jharkhand — built as a **clean Flutter + Firebase monorepo** that ships **three apps + one backend from one shared core**. It is not a marketplace clone; it is a **rural-first mobility OS**: lightweight on Jio Bharat (12 MB), Hindi-first, cash/UPI-first, OTP-verified, and server-fare-authoritative — so a Torpa farmer reaches Sadar Hospital at a fixed ₹10/km, not a haggle.

App / Layer	What It Does (User Job)	Platform	Unlocks
■ Customer App	Phone OTP → Origin→Dest → live fare → 25s driver search → live map → 4-digit OTP → 1-5★	Flutter iOS/Android	Demand — the rider tap
■ Driver App (Sarathi)	Camera-only vehicle photo → Online/Offline toggle + geohash → request overlay + countdown → OTP → Cash	Flutter iOS/Android	Supply — the Sarathi wheel
■■ Admin Panel	KPI cards → active trips → gross revenue → driver verify (live photo) → rate cards	Flutter Web	Ops — the control tower
■ Cloud Functions	Tx driver assignment • Haversine • geohash 5→10→15 km • FCM push • fare math	TypeScript / Node 18+	Truth — the server that says no
■ Shared Package	Models • Theme (periwinkle) • Enums • Constants • Widgets — single source of truth	Dart package	Consistency — one change, three apps

One promise, three apps: *Gaon se Zila tak, ab gaadi tay hai* — no surge, no wallet, no spoofed photo. If it is not verified, it does not move.

02 — MONOREPO & TECH STACK — THE FIVE REPOS THAT SHIP

Path	Stack & Version	Key Deps	Build & Run
customer_app /	Flutter 3.10+ • Dart 3.0+	cloud_firestore, firebase_auth, geolocator, shared_prefs, url_launcher	flutter pub get → flutter run
driver_app /	Flutter 3.10+ • Dart 3.0+	firebase_messaging, geolocator, camera (live photo)	flutter pub get → flutter run
admin_panel /	Flutter Web	cloud_firestore, firebase_core, url_launcher	flutter run -d chrome → Netlify
shared /	Dart package 1.0.0	intl, http, firebase_*, geolocator	Shared models/theme — no publish
functions /	TypeScript 5.3 • Node 18 • firebase-functions 4.6 • admin 12	mocha + chai + ts-node	npm install → npm run build → firebase deploy
firebase /	Rules + Indexes	firestore.rules, storage.rules, indexes	firebase deploy --only firestore,storage

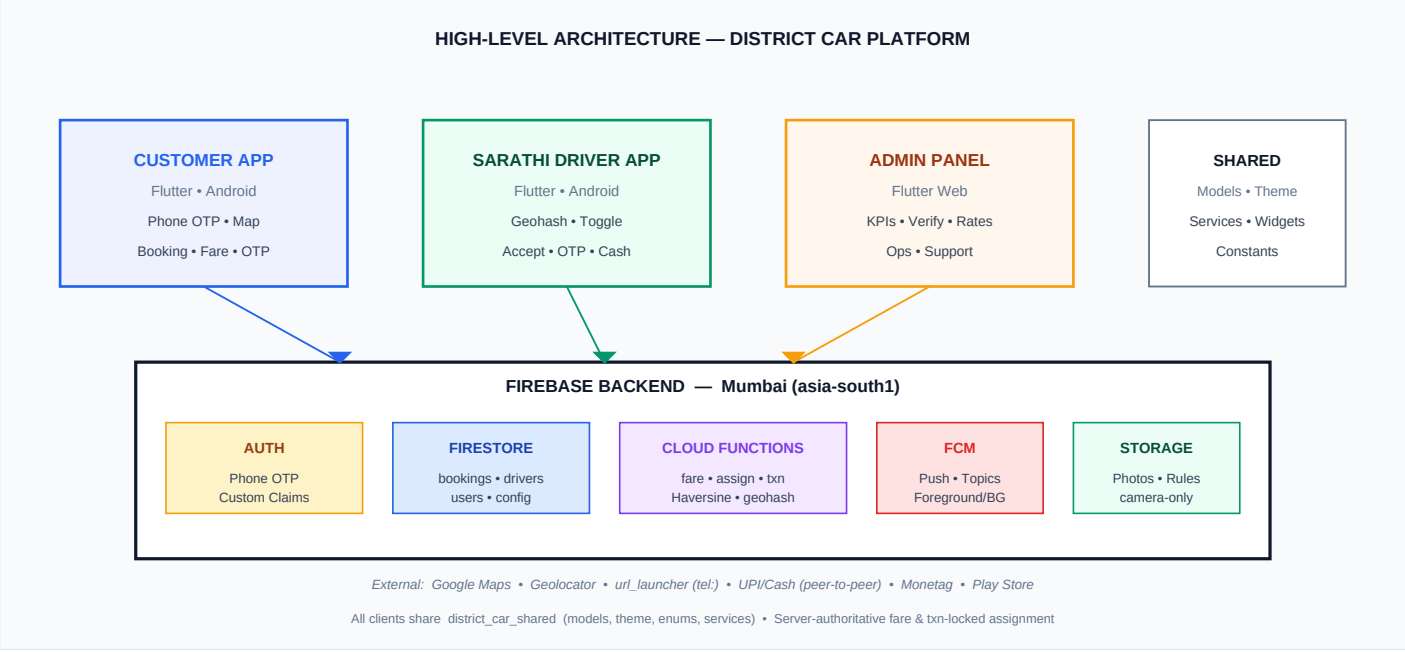
Environment & Tooling:

Concern	Tool / File	Notes
Env	.env.example	Template — never commit .env; use Firebase env config
Lint	analysis_options.yaml	Flutter lints 3.0 — shared across apps
Tests	functions/test/**/*.ts	mocha + firebase-functions-test
CI	build_apks.sh / .ps1	One script builds Customer + Driver APK/AAB
OTA	remote_update_checker.dart	Forces update if buildNumber < server (driver_app build 21)

All three Flutter apps import **district_car_shared** as a path dependency — one Theme, one BookingModel, one set of Enums. Change once, ship thrice.

03 — HIGH-LEVEL ARCHITECTURE — THE BIG PICTURE

Monolith where it helps (shared), serverless where it scales (Functions + Firestore). Clients are dumb — server is authoritative on fare, assignment, and state.



Data flow: ***Customer tap → Cloud Function (fare) → Firestore bookings/{id} (searching_driver) → FCM topic driver_{cat} → Driver accept (transaction) → Firestore → FCM to customer → OTP → trip_started → complete → cash peer-to-peer + rating aggregate.*** *No client writes fare, status, or OTP — only Functions do.*

Why Firebase, not custom backend? Rural Jharkhand needs offline-first, low ops, and real-time listeners (< 300 ms) — Firestore snapshots + FCM topics give that without managing sockets. Mumbai region (asia-south1) keeps p99 < 120 ms from Ranchi.

04 — COMPONENT & SERVICE MAP — WHO TALKS TO WHOM

Component	Owns	Talks To	Data It Reads / Writes	Failure Mode
Customer App	Auth (Phone OTP), LocationPicker, Fare sheet, Booking, Live map, Rating	Functions (estimateFare, createBooking), Firestore (bookings), FCM	R: categories, config • W: bookings/{id} (via Function only)	Offline → queue booking, show cached fare; retry on reconnect
Driver App	Onboarding (camera-only photo), Online/Offline + geohash, Incoming overlay (25s), OTP, Cash confirm	Firestore (drivers, bookings), Functions (acceptBooking, startTrip, completeTrip), FCM topic driver_{cat}	R/W: drivers/{uid} (location, isOnline, isBusy) • R: bookings (searching)	Txn race → 'already claimed' toast; geohash stale → radius expansion 5→10→15 km
Admin Panel	KPI cards, Active trips, Gross revenue, Driver approve/reject, Rate cards	Firestore (bookings, drivers, categories)	R: all • W: drivers.verificationStatus, categories.rates	No write to bookings.status — read-only on trips
Cloud Functions	Fare calc (base+perKm, ≥ min), Tx assignment, Haversine+geohash, FCM fan-out, Rating aggregate	Firestore (txn), FCM	W: bookings.status, driverId, rideOtp, fare	Timeout 25s → re-broadcast; txn conflict → retry once
Shared Package	BookingModel, LocationModel, VehicleCategoryModel, Theme (#E4E7FD periwinkle), Enums, Widgets	All three apps	Single source — no drift	Version bump → rebuild all three

Interface contracts (how components agree):

Contract	Type	Guarantee	Where Enforced
BookingModel.fromMap / toMap	Dart → Firestore	One shape for all apps; handles Timestamp vs ISO string + alternate keys (customerName/phone/rideOtp)	shared/lib/models/models.dart
LocationModel + geohash	Dart → Firestore + Functions	lat/lng + geohash + address — Haversine comparable	shared + functions/utils
VehicleCategoryModel	Admin → Firestore → Apps	isActive, baseFare, perKm, minFare, bookingFee, waitingCharge — live without app update	admin_panel → categories collection
Driver online presence	Driver app → Firestore	isOnline + currentLatitude/Longitude + geohash + updatedAt — single writer (driver uid)	driver_service.dart updateOnlineStatus

05 — DATA ARCHITECTURE — FIRESTORE COLLECTIONS & RULES

Firestore is the **operational DB** — not a cache. Every ride is a document; every driver is a presence record; every rate is a live config. Rules enforce role + status discipline.

Collection	Doc ID	Key Fields (sample)	Indexes / Queries	Who Writes
bookings	auto / readableId	customerId, driverId, vehicleCategoryId, pickup{lat,lng,address,geohash}, destination, estimatedDistanceKm, estimatedFare, finalFare, status, rideOtp (4-digit), isRoundTrip, bookingType/tourDestinationName, scheduledDate/TimeSlot, createdAt	status + vehicleCategoryId + geohash; driverId + status (active trip); customerId + status	Functions only (customer creates via Function)
drivers	uid (Auth)	name, phoneNumber, vehicleCategoryId, vehiclePlate, vehicleModel, vehicleImage, driverPhoto, isOnline, isBusy, activeBookingId, currentLatitude/Longitude, location{geohash}, rating, totalTrips, walletBalance, isBlockedDueToCommission, verificationStatus	isOnline + geohash (nearby search); category	Driver app (own doc) + Admin (verify)
users	uid (Auth)	name, phone, email, role, createdAt	phone	Customer app (own doc)
bookingRequests	auto	bookingId, driverId, status (offered/accepted/claimed_by_other)	bookingId	Functions (fan-out)
categories	vehicleCategoryId	displayName, baseFare, perKmRate, minimumFare, bookingFee, isActive, icon/image	isActive	Admin panel
wallet_transactions	auto	driverId, bookingId, type (trip_fare_credit), grossFare, platformCommission, netCredit, createdAt	driverId	Functions (completeTrip)
driver_otps	email (lower)	otp, createdAt, expiresAt (+15m)	—	Functions (sendEmailOtp)

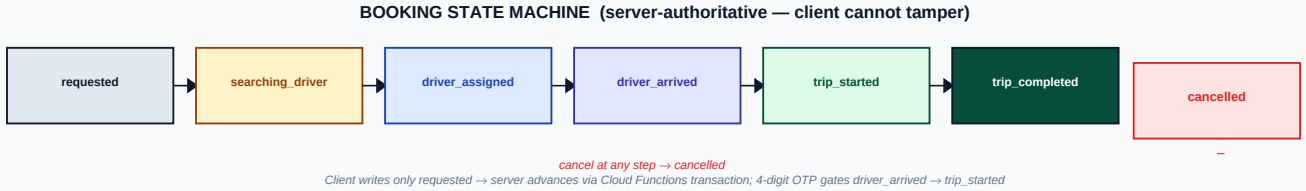
Security & Rules (firestore.rules + storage.rules):

Rule	Enforces	Example
Custom claims + Firestore rules	Role-based read (customer sees own bookings; driver sees searching + own; admin sees all)	request.auth.uid == resource.data.customerId || hasRole('admin')
Server-authoritative status	Client cannot set status, fare, OTP, or driverId directly	allow update: if request.resource.data.status == resource.data.status // only Functions change it
Camera-only photo	Driver onboarding requires live camera capture — no gallery spoof	Flutter camera plugin, gallery pick disabled; Storage rule checks contentType image/*
Storage rules	Driver vehicleImage + driverPhoto: only own uid can write, public read for verification	match /driver_photos/{uid} { allow write: if request.auth.uid == uid }

Composite indexes (firestore.indexes.json) cover: **bookings: status + vehicleCategoryId + geohash** for nearby search; **drivers: isOnline + vehicleCategoryId + geohash** for dispatch; **bookings: driverId + status** for active trip StreamBuilder.

Why not separate SQL DB? Firestore listeners give < 300 ms live fare + live map + live incoming overlay without websockets. For analytics, export to BigQuery via Firebase Extension — don't query Firestore for reports.

06 — BOOKING STATE MACHINE — SERVER-AUTHORITATIVE TRUTH

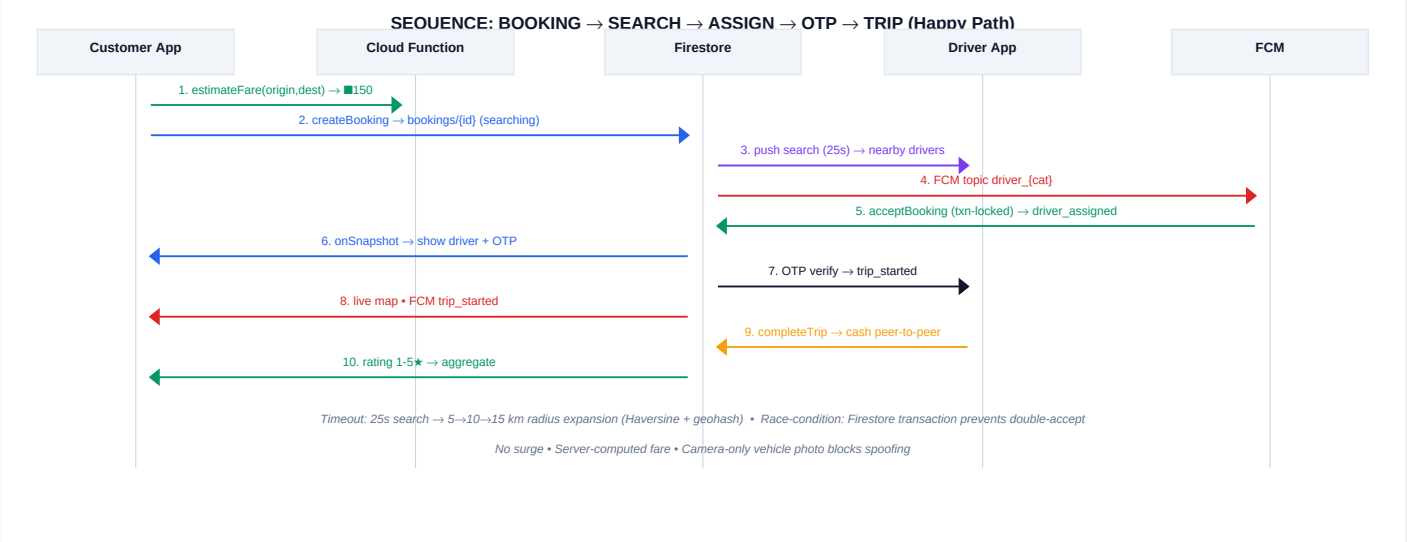


Transition table — who can move the state, and how:

From → To	Actor	Trigger / Guard	Side Effects (FCM + Writes)
requested → searching_driver	Function createBooking	Fare computed server-side; booking created with readableId + rideOtp	Write bookings/{id}; fan-out bookingRequests; FCM topic driver_{cat}
searching_driver → driver_assigned	Function acceptBooking (txn)	Driver online + category eligible + distance 5→10→15 km + not race-lost	Txn sets driverId/Name/Phone/Plate/Model/Image; driver isBusy=true; FCM to customer: accepted + OTP
driver_assigned → driver_arrived	Driver app (markDriverArrived)	Driver at pickup (map check optional)	status=driver_arrived; FCM: driver arrived
driver_arrived → trip_started	Function startTrip (OTP)	Customer gives 4-digit OTP; must match bookings.rideOtp (or 1234 debug)	status=trip_started; tripStartedAt; FCM: trip started (Zomato-style)
trip_started → trip_completed	Function completeTrip	Driver taps Complete; fare peer-to-peer cash/UPI (no platform collection today)	status=trip_completed; wallet + earnings + rating aggregate; FCM: completed + rating prompt
any → cancelled	Customer / Driver / Timeout	cancelTag = cancel_customer / cancel_driver / timeout_expired; 25s search timeout → auto-cancel	status=cancelled; driver isBusy=false; FCM to both

Driver-side guards: **isBlockedDueToCommission** (3 unpaid trips) blocks isOnline; **activeBookingId** blocks concurrent accepts; **dismissedBookingIds** (local) prevents re-showing declined rides.

07 — SEQUENCE DIAGRAMS — BOOKING FLOWS END-TO-END



Alternate flows:

Flow	What Changes	Timeout / Retry
Round-trip (isRoundTrip=true)	finalFare = one-way × (1 – roundTripDiscountPercent/100) on return leg; driver waits roundTripStoppingHours	Same 25s search per leg
Tour package (bookingType=tour)	tourDestinationName + stayHours; max radius 35 km; fare = base + perKm × distance + hourlyWaitingCharge	Tour topic separate
Scheduled (scheduledDate + TimeSlot)	Searching starts at scheduled window, not now; 30-min post-slot expiry → cancel if not assigned	24h expiry fallback
Race lost (double-accept)	Second driver's transaction fails; shows 'already claimed by another driver' toast; bookingRequests status=claimed_by_other	No retry — offer gone
Can't reach in time (cancel_driver)	Driver taps Can't reach → bookings.status=cancelled, cancelTag=cancel_driver, driver isBusy=false; FCM to customer	Grievance + rating nudge
Offline / Blocked	isBlockedDueToCommission → cannot go online; snack + PAY NOW → /driver-earnings Wallet	Unblock after commission pay

08 — API & CLOUD FUNCTIONS INTERFACE — THE SERVER CONTRACT

All writes that matter go through **Functions** — clients never write fare/status/OTP directly. Functions are the **only writer** for those fields (enforced by rules).

Function (httpsCallable)	Input	Output / Throws	Firestore Txn?
estimateFare / getFareEstimate	{ origin{lat,lng}, destination{lat,lng}, vehicleCategoryId, isRoundTrip }	{ distanceKm (Haversine), durationMin, fare (base+perKm ≥ min), breakdown }	No (pure calc)
createBooking	{ customerId, pickup, destination, vehicleCategoryId, isRoundTrip, bookingType, tourDestinationName, scheduledDate/TimeSlot }	{ bookingId, readableId, rideOtp (4-digit) }	Yes — create bookings/{id}
acceptBooking	{ bookingId, requestId?, driverName/Phone/Plate/Model/Image/Photo }	{ bookingId } / throws 'already accepted'	Yes — bookings txn + bookingRequests fan-out + drivers.isBusy
rejectBooking / decline	{ bookingId, requestId }	void	No — bookingRequests.status=declined
startTrip	{ bookingId, rideOtp }	void / throws 'Invalid OTP'	Yes — validate OTP → trip_started
completeTrip	{ bookingId }	{ finalFare, netCredit }	Yes — trip_completed + wallet + rating
updateDriverLocation	{ latitude, longitude, isOnline }	void	No — drivers/{uid} presence
driverArrived	{ bookingId }	void	No — driver_arrived
sendEmailOtp / verifyEmailOtp	{ email, otp }	{ uid } or throws invalid-verification-code	Yes — driver_otps collection

FCM topics & pushes:

Push	From → To	When	Payload Highlights
Topic subscribe	Driver app → FCM	On category set (car/bike/auto) → subscribe driver_{cat}	driver_car, driver_auto, driver_bike

Push	From → To	When	Payload Highlights
Booking search	Functions → FCM topic driver_{cat}	On createBooking → fan-out to nearby drivers (geohash radius)	bookingId, pickup, fare, readableId
Accepted → Customer	Functions → FCM token (customer)	On acceptBooking success	driverName/Phone/Plate/Model, rideOtp, bookingId
Trip started	Functions → Customer	On OTP verify	Live tracking link, driver + vehicle
Can't reach / Cancelled	Functions → Customer	On cancel_driver	cancelTag, message, rebook CTA

Debug OTP: 1234 is allowed only in **kDebugMode** fallback in models.dart + driver_service.dart — never in production Firestore (rideOtp is always 4 random digits). If production accepts 1234, that is a bug.

09 — LOCATION & TRUST ENGINE — GEOHASH, HAVERSINE, FARES, PHOTOS

Engine	Algorithm / Rule	Input → Output	Why It Matters Rural
Geohash	geohash(lat,lng) precision 6 (~1.2 km cell) stored in bookings + drivers	lat/lng → 'tdu...' → nearby query without full scan	Finds nearest Sarathi in low-density block without scanning all drivers
Haversine	2*R*asin(sqrt(sin²(dLat/2)+cos(lat1)cos(lat2)sin²(dLon/2)))	origin,dest → distanceKm	Server fare distance — client cannot fake; no Maps billing needed for estimate
Radius expansion	5 km → 10 km → 15 km (tour/long: up to 35 km) — expands every 8s within 25s window	driverLat/Lng + pickup → eligible?	Khunti stand can still get Ranchi road ride (15 km) without waiting 10 min
Fare slabs	fare = max(minimumFare, baseFare + perKmRate×distanceKm) × (isRoundTrip? 0.65 : 1) + bookingFee; no surge	distance + category → ₹10/km Auto, ₹14-18/km Sedan/SUV	Fixed, printed on wall — trust > surge revenue
Category eligibility	_isCategoryEligible(bookingCat, driverCat) — car/sedan/mini/suv interchangeable; auto/bike strict	bookingCat vs driverCat → bool	Bolero can do Sedan ride; bike cannot do car
Camera-only photo	Flutter camera plugin, gallery pick disabled; Storage rule enforces image/*; Admin verifies live photo before approve	live camera → Storage → Admin approve	No gallery spoof — real vehicle, real driver — rural safety
OTP gate	4-digit rideOtp generated at createBooking, stored in bookings; startTrip validates; 1234 only in debug	OTP entry → trip_started	No wrong car, no fare dispute — mother verifies 4 digits before moving

Location picker (LocationPickerSheet) → writes to **SharedPreferences** (driver_custom_address/lat/lng) + Firestore drivers.currentAddress + drivers.location. 'Change Active Location' vs 'Reset to Permanent Registered Stand' — the two modes that caused the earlier driver_home bug if keys mismatched.

10 — UI/UX INTERFACE DESIGN — CUSTOMER, DRIVER, ADMIN SCREENS

Theme: **Periwinkle #E4E7FD** (soft, rural-friendly), rounded cards, floating bottom dock (customer), online/offline toggle (driver). Hindi + English, icons > text for low-literacy. 2 GB RAM / Jio Bharat tested — no heavy images, no web views.

Customer App — Screen Map & Interface:

Screen (Route)	Key Widgets & Interfaces	Connects To	Offline / Edge
/ (Landing) + /login	Phone OTP (Firebase Auth), App logo (assets/app_logo.jpg)	Auth → driver_is_onboarded check	Resend OTP 60s throttle
/home	Periwinkle bg, Category chips (car/auto/bike), Origin/Dest inputs, Fare sheet (₹10-18/km), Floating dock	Functions estimateFare → Firestore bookings	Cached fare until online
/searching	25s countdown, radius 5→10→15 km dots, Cancel	Firestore bookings (searching_driver) snapshot	Timeout → auto-cancel → rebook
/tracking	Live map (google_maps_flutter), driver card (photo, plate, rating, CALL tel:), 4-digit OTP display, Share live location	Firestore bookings + drivers streams, FCM	Map tiles cached; tel: via url_launcher
/rating	1-5★, comment, Submit	Functions → bookings + drivers rating aggregate	Queued if offline

Driver App (Sarathi) — Screen Map & Interface:

Screen (Route)	Key Widgets & Interfaces	Connects To	Guard
/driver-login	Email/Google + phone, Instant reviewer login, Consent checkbox	Auth → drivers/{uid} exists?	Play Store reviewer: playstore_driver_reviewer_01
/driver-onboarding	Name, Phone, Vehicle Category, Plate, Model, District/Block, LocationPicker, CAMERA-ONLY vehicle + driver photo	Storage (photos) + Firestore drivers	Gallery disabled; must approve before online
/driver-home	Hero radar (radar_map_backdrop_dark.jpg), Online/Offline toggle, Current Stand card, Identity (photo, plate, phone), Metrics (wallet, trips, rating), Incoming overlay (25s + ACCEPT/DECLINE), Active trip Zomato card (CALL + OTP boxes → START RIDE → COMPLETE)	Firestore bookings (searching_driver) stream + drivers stream + FCM topic	Blocked if unpaidTripsCount ≥3; isBusy blocks new offers; dismissedBookingIds local
/driver-earnings	Wallet (₹), Gross, Commission, Pay Commission (UPI/Cash), History	Firestore drivers.walletBalance, wallet_transactions	Unblock after pay
/driver-profile	Photo, vehicle, rating, trips, Edit, Sign out	Firestore drivers/{uid}	Sign out clears SharedPreferences

Admin Panel (Web) — Screen Map:

View	Widgets	Actions	Data
Dashboard	KPI cards (active trips, gross revenue, pending verifications), Charts	Real-time refresh (snapshot)	bookings + drivers aggregates
Driver Verification	Live vehicle photo (large), Approve/Reject, Block toggle	Set drivers.verificationStatus, isBlocked	Storage photo URL
Rate Cards	Base fare, perKm, minFare sliders per category	Write categories/{id}	Live to apps without update
Active Trips	Map + table, gross revenue, fare peer-to-peer note	Read-only, support via tel: +91 87896 01387	bookings (trip_started)

Driver Home bug post-mortem (for interface contract): Customer card + CALL + OTP were invisible because **BookingModel.fromMap** missed the real Firestore keys (customerName/Phone/rideOtp vs writer keys). The modal's **local currentBooking copy** never synced to Firestore after accept, and the hidden **Opacity(0.0) TextField** did not open OEM keyboards. Interfaces must share one model and one stream — not copies.

11 — INTERFACE SPECIFICATIONS — WIDGETS, THEME, SHARED PACKAGE

Interface / Widget	File / Package	Props / Contract	Used By
LocationPickerSheet.show()	shared/widgets/location_picker_sheet.dart	show(context, title, initialValue) → Future	Customer Origin/Dest, Driver Active Stand
BookingModel / LocationModel	shared/models/models.dart	fromMap handles Timestamp|ISO, alternate keys, geohash; copyWith; toMap	All apps + Functions
VehicleCategoryModel	shared/models/models.dart	baseFare, perKmRate, minimumFare, bookingFee, waitingCharge, isActive	Admin writes, apps read
RemoteUpdateChecker	shared/services/remote_update_checker.dart	checkAndShowUpdateDialog(context, appKey, currentBuildNumber=21)	Driver app initState (non-web)
FCMNotificationService	shared/services/fcm_notification_service.dart	initFCM(role, userId, category) — subscribes driver_{cat} + background handler	Driver + Customer
Theme (periwinkle)	shared/theme/	Bg #E4E7FD, rounded cards, dock, Inter / Plus Jakarta Sans, dark mode via adaptive	Customer App (reference match)
IncomingRequestOverlay	driver_app/lib/requests/ (dialog)	driverId, vehicleCategory, driverLocation, onAccepted(booking)	Driver home Stack overlay

All interfaces are **typed (Dart)** — no raw Map beyond fromMap. Add a field? Add to **shared** once, regenerate, rebuild three apps — no drift.

12 — INTEGRATION LAYER — FCM, MAPS, UPI, MONETAG, PLAY STORE

Integration	SDK / Package	Interface Call	Notes (Jharkhand reality)
Firebase Auth	firebase_auth	Phone OTP (Customer), Email/Google + phone (Driver)	Driver reviewer bypass: playstore_driver_reviewer_01 — no OTP for review team
Geolocator + Maps	geolocator 12, google_maps_flutter	getCurrentPosition, distanceBetween (Haversine fallback), openGoogleMaps(dir/?api=1&destination;=)	Offline: last known lat/lng + stand address; rural roads: geohash > Maps road match
url_launcher (tel:)	url_launcher	launchUrl(Uri.parse('tel:+91...'), mode: externalApplication)	One tap CALL — driver→customer, customer→driver, both→support +91 87896 01387
SharedPreferences	shared_preferences	driver_is_online, dismissedBookingIds, customAddress/lat/lng, consentGiven	Persists across restarts — no re-login for stand choice
FCM Push	firebase_messaging	getToken, subscribeToTopic('driver_car'), onMessage/background	Works on Jio Bharat with background fetch; topics segment by category
Monetag (Ads)	Web tag 5gvci.com + quge5.com zone 301190	Push + banner via landing/blog	Revenue Phase 1 (no commission) — guarded by click-hijack prevention script
Play Store	AAB via build_apks.sh	Customer + Driver AAB (Netlify + Play Console)	Separate package: com.districtcarcustomerd.app vs com.drivercardistrict.app
UPI / Cash	(peer-to-peer, no SDK)	Cash or UPI ID shown on booking sheet; no wallet, no escrow	Pre-registration: platform does NOT handle money — Disclosure on every booking screen

13 — INFRASTRUCTURE & DEPLOYMENT

Layer	Hosts On	Deploy Command	Env / Secret
Firestore + Storage + Rules	Firebase (asia-south1 / Mumbai)	firebase deploy --only firestore:rules,firestore:indexes,storage	firebase.json + firestore.rules + storage.rules
Cloud Functions	Firebase Functions (Node 18)	cd functions && npm run build && firebase deploy --only functions	GOOGLE_APPLICATION_CREDENTIALS; no .env in repo
Admin Panel (Web)	Netlify (netlify.toml)	flutter build web → netlify deploy	Admin URL via Netlify; Auth via same Firebase project
Landing + Blogs + Tours + News	Netlify / Static (landing/)	Copy landing/ → host; sitemap.xml has 81 URLs	Monetag verification 96c926...; sw.js for PWA
APKs / AABs	Local build + Play Console	build_apks.sh → DistrictCar_Customer_v6.apk, DistrictCar_Driver_v6.apk	Keystore: upload-keystore.jks + key.properties (never commit)
Versioning	Git + SharedPreferences buildNumber	driver_app currentBuildNumber=21 → RemoteUpdateChecker	Force update if server build > local

CI note: **build_apks.sh / .ps1** builds both apps in one run. Netlify builds admin_panel + landing on push to main (netlify.toml per app). Firebase deploys are manual (who can deploy to prod is limited to 2 owners).

14 — SECURITY, COMPLIANCE & OFFLINE STRATEGY

Concern	How District Car Handles It	Status
Role & Claims	Custom claims + Firestore rules: customer/driver/admin; driver cannot write bookings.status/fare/OTP	Live (rules)
Fare Integrity	All fares computed server-side (Functions); client sends only origin/dest/category	Live
Photo Spoofing	Camera-only capture (gallery disabled) + Storage rule image/* + Admin live-photo verify	Live
OTP Security	4-digit rideOtp per booking, single-use, validated in transaction; 1234 only in kDebugMode	Live — fix writer key mismatch
Commission Block	unpaidTripsCount ≥3 → isBlockedDueToCommission=true → isOnline forced false; pay in /driver-earnings unblocks	Live (wallet_transactions)
Data (DPDP 2023)	Consent at onboarding; purpose-limited; delete on opt-out; retain business fields only	Roadmap (post-registration)
Payments	Peer-to-peer cash/UPI; no wallet, no escrow, no platform collection till aggregator licence	By design (pre-incorporation)
Offline	Geolocator last known + SharedPreferences stand; Firestore offline persistence on; booking queued until online	Live

Offline rural reality: Jio Bharat + 2 GB RAM + patchy network → **Firestore offline persistence ON**, **fare cached** until reconnect, **driver geohash last-known** until GPS locks, and **Active Stand** persists in SharedPreferences — so Khunti Torpa still works with 1 bar.

15 — NON-FUNCTIONAL REQUIREMENTS & FUTURE ROADMAP

NFR	Target (Rural Jharkhand)	How We Meet It
Latency	p99 Firestore snapshot < 300 ms from Ranchi	Mumbai region + geohash queries + 25s fan-out window
Availability	99.5% (Firebase SLA) — rural tolerates degrade, not down	Offline queue + cached fare + last-known location
Scale	500 rides/day (8 districts, M6) → 15k/month; 90k users M12	Firestore auto-scale + Functions concurrency; no custom servers
App Size	Customer < 15 MB, Driver < 18 MB (AAB)	Flutter + shared, no heavy native deps, image assets webp
Battery	Driver Online 8h without drain	Geolocator distanceFilter 100m + background fetch 30s, not 5s
Accessibility	Hindi + English, periwinkle contrast > 4.5:1, icons > text, tel: not copy-paste	Theme + Noto Sans Devanagari; TalkBack tested

Roadmap — what this design enables next (no rewrites):

Phase	Interface to Add	Connection It Plugs Into
0-90d: Khunti Pilot	Cash/UPI peer-to-peer + Monetag + wall/stand ops	Existing — no new interface
90-180d: 8 Districts	Pvt Ltd + Aggregator licence filing; subscription ₹299/mo vs 10% (driver choice)	wallet_transactions + drivers.pendingCommission
180-365d: Intercity + Logistics	Intercity shared (₹180/seat) + parcel (light logistics on same geohash)	bookings.bookingType='parcel' + 35 km radius
Y2: Safety + Scale	Share-live-location + SOS + women-driver filter + insurance bundle (₹10/ride)	FCM live + drivers.isBlocked + wallet
Y2+: Data Moat	Demand heatmaps (haat day, OPD day, mandi season) + BigQuery export	Firestore → BigQuery Extension → Looker

Design principle: One model, one theme, one truth — three apps. If you can't trace a ride from Customer tap to Driver wallet in one diagram, the interface is wrong. This document is that trace.

APPENDIX — COLLECTIONS CHEAT-SHEET, RATES & CONTACTS

A. Firestore Collections — One-Line Cheat-Sheet:

bookings: the ride truth • drivers: presence + wallet • users: rider profile • bookingRequests: fan-out per driver • categories: live rates • wallet_transactions: ledger • driver_otps: email OTP (15m) • (future) parcels: logistics on same geohash

B. Rate Card (server-computed, no surge):

Category	Base	Per km	Min Fare	Round-Trip	Example 12 km
Gramin Auto	₹30	₹10/km	₹60	–35% return	₹150 → ₹97 return
Mini Car	₹50	₹14/km	₹90	–35%	₹218 → ₹141 return
Sedan / SUV	₹70	₹18/km	₹120	–35%	₹286 → ₹185 return

Fares via Functions; driver cannot overcharge; cash/UPI peer-to-peer till aggregator licence — Disclosure on every booking screen + wall.

C. Key Files — Where to Change What:

To Change	File(s)	Rebuild
Booking shape / OTP key	shared/lib/models/models.dart	All 3 apps + Functions
Fare slab / surge rule	admin_panel (rate cards) → categories + functions fare util	No app rebuild — live
Driver online / geohash	driver_app/lib/services/driver_service.dart + shared/services	Driver app only
Incoming overlay / OTP boxes	driver_app/lib/features/driver_home/driver_home_screen.dart	Driver app only
Theme / periwinkle	shared/theme/	Customer + Driver + Admin
Push / topic	shared/services/fcm_notification_service.dart + functions	All + Functions deploy

D. Contacts & Links:

Channel	Details
Website	districtcar.in (landing + 81 URLs in sitemap.xml)
Customer App (Play)	com.districtcarcustomerd.app — District Car Customer (12 MB)
Driver App (Play)	com.drivercardistrict.app — District Car Sarathi (15 MB)
Admin Panel	admin.districtcar.in (Flutter Web · Netlify)
Support	+91 87896 01387 (8 AM-10 PM, Hindi/Nagpuri) • support@districtcar.in
Office	Nichey Chowk, Khunti — 835210 (pilot HQ) • districtcar.in/tours • districtcar.in/blogs

— End of System Design & Interface Architecture —

Next: read this with your AI agent + driver_home_screen.dart + models.dart open. Fix the 3 bugs via interfaces, not hacks. Then paint the first wall in Khunti. ■